



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

METODOLOGÍA BASADA EN IA PARA LA MIGRACIÓN DE CÓDIGO LEGADO: SECTOR BANCARIO

Trabajo de investigación/grado para optar el título de
Magister en Ciencias de la Información y las Comunicaciones

José Antonio Silva Rojas

Asesor

Nelson Enrique Vera Parra

Bogotá D. C. – Colombia

Mayo de 2025

Contenido

1. Grupo de Investigación	4
2. Línea de Investigación	4
3. Director.....	4
4. Resumen del proyecto	4
5. Presentación de la propuesta	5
6. Formulación del problema	6
6.1 Planteamiento del problema.....	6
6.2. Formulación del problema	7
6.3. Sistematización del problema.....	7
7. Marco teórico o conceptual	7
7.1 Marco Conceptual	7
7.1.1 Código legado	7
7.1.2 Sintaxis y semántica en código	8
7.1.3 Large Language Model (LLM)	8
7.1.4 Pruebas	9
7.1.5 Arquitectura limpia	9
7.2 Marco Teórico	10
7.2.1 Investigaciones previas	10
7.3 Marco espacial.....	13
7.3.1 Sector Bancario	13
7.3.2 Rol Inteligencia Artificial en el sector bancaria.....	13
7.3.3 Sistemas de integración.....	14
7.3.3.1 IBM Integration Bus versión 10 (IIB10) y App Connect Enterprise (ACE)	15
7.3.4 Scaffold	16

8.	Objetivos	17
8.1	Objetivo general	17
8.2	Objetivos específicos.....	17
9	HIPÓTESIS	18
10	METODOLOGÍA	18
10.1	Creación de prompts código legado	18
10.2	Creación prompts lenguaje a migrar	37
10.3	Evaluación modelos para la migración de código.....	40
11	RESULTADOS Y DISCUSIÓN.....	41
12	Bibliografía.....	42

1. Grupo de Investigación

Grupo de investigación GICOGE (Grupo Internacional en Informática y Comunicación para la Gestión de Conocimiento), creado en el 2001. Fundado por el profesor José Nelson Pérez y con clasificación A1.

2. Línea de Investigación

Calidad e Innovación en Informática, Comunicaciones y Gestión del Conocimiento para el Desarrollo Organizacional

3. Director

Dr. en Ingeniería Nelson Enrique Vera Parra. Ms en Ciencias de la Información y las Comunicaciones e Ingeniero Electrónico.

4. Resumen del proyecto

El código legado, consiste en código funcional, pero con obsolescencia; lo que genera inconvenientes de escalamiento y mantenibilidad, creando brechas de seguridad y soporte. Es por ello que, el objetivo de la presente investigación es desarrollar una metodología para definir la migración de código legado, conservando su funcionalidad y semántica. Para ello, se identifica aquellos estudios previos rescatando de ellos sus aportes y limitantes, siendo una línea base de la investigación. Además, como insumo, se abordó una de las tecnologías que se emplean en la capa de integración a nivel bancario y, la cual, tuvo soporte hasta el 2025. De allí, se extrayeron los repositorios que sirvieron para generar los prompts y la base de datos. De igual forma, para la estandarización hacia el código migrado, se propuso uno de los patrones más usados a nivel mundial conocido como Clean Architecture.

A partir de este punto, se procede a definir una metodología, partiendo de: la obtención y limpieza del conjunto de datos, validación de modelos, definición y diseño de reglas, definición de métricas de evaluación y extracción de patrones. Con estos recursos, se determinó un proceso de migración

empleando LLM y, por último, se verificó la métrica para evaluar el código migrado. Todo lo anterior, desempeñando un papel fundamental para estructurar la metodología en cuestión.

5. Presentación de la propuesta

El código legado es un hecho recurrente y presente en las empresas a nivel mundial, en particular en el sector bancario, esto ocurre debido a que, el tiempo de vida promedio de un servicio o API, suele tener en promedio dos a cinco años. Al cabo de este tiempo, el código inicial no ha sufrido mayores cambios y el personal o empresa que arrancó con su desarrollo, posiblemente ya no se encuentran en la compañía y súmele a este hecho, la poca o nula documentación y set de pruebas con la que partió el desarrollo y la falta de mantenibilidad o escalabilidad, entonces, acá se enfrenta con otro gran hito y es lo vertiginosamente cambiante que es el software, alrededor de 6 meses a un año; causando que ese código inicial quede postergado y amarrado en una tecnología obsoleta.

Es acá donde la inteligencia artificial (IA) ha buscado mitigar inconvenientes en muchos aspectos del software, entre ellos, se ha enfrentado a la refactorización de código, automatización de pruebas, asistente de desarrollo y, un poco más relegado, la migración de código. Arrancando de este último ítem, se enfoca la siguiente propuesta de investigación, dado que, la migración de código legado es un tema que los desarrolladores busca evadir, debido a su complejidad, tiempos de ejecución, generación de pruebas, documentación, entre otros.

En este caso, se propone una metodología que abarque un conjunto estructurado y procedimental para la migración de código legado, manejando una rama del lenguaje de procesamiento natural (NLP), conocida como: Modelo de Lenguaje Grande o LLM. Esta metodología confrontará varios modelos de LLM y herramientas conocidas en el mercado y validará a partir de un grupo de pruebas, la eficacia de cada uno de ellos, diseño, definiendo y reentrando reglas y modelos para dicho fin. Como recurso para hacer estas evaluaciones, se contará con un repositorio de los ejemplos de una herramienta deprecada este año, empleada para la integración de servicios bancarios y, como resultado, se busca migrar el código a un enfoque de arquitectura limpia (clean architecture), dado el auge de este concepto a nivel mundial, además, esto estará soportado sobre el lenguaje Java.

6. Formulación del problema

6.1 Planteamiento del problema

Se entiende por código legado en la industria como aquel código anticuado, trasladado de otros equipos o desarrolladores y que es difícil de modificar y de poco entendimiento, además, cuenta con pruebas limitadas o nulas y, todavía, se encuentra en uso. (Feathers, 2004) Siendo en algunos casos el eje central de sus sistemas y con una alta dependencia en plataformas desarrolladas hace décadas.

Esto trae consigo, lentitud en la implementación de nuevas funcionalidades, costos de mantenimiento y operación, incompatibilidad en despliegues en tecnología recientes, fallas de seguridad, entre otras. Esto es producido, por la falta de documentación, pruebas o resistencia a la actualización o cambio por parte de la industria, en la mayoría de casos por temas de costos o impactos en sus operaciones financieras (Bennett & Rajlich, 2000)

Lo anterior genera una vulnerabilidad tanto técnica como estratégica, representada en:

1. Ciberataques, debido a las pocas o nulas actualizaciones que puedan presentar creando brechas por falta de soporte. En su reporte anual del 2023, la Superintendencia Financiera de Colombia (SFC), reporta más de 36.000 millones de ciberataques (Colombia, 2025)
2. Aumento de costos, debido a que su intervención se debe hacer con personal especializado, quienes realizan parches continuos, incrementando cada vez más su complejidad y entendimiento.
3. Riesgos de fallos críticos en procesos transaccionales que pueden acarrear sanciones por parte de la SFC, quien respaldada por la LEY 1328 DE 2009, establece protección de los consumidores financieros en las relaciones entre estos y las entidades vigiladas. (República, 2022)
4. Incursión limitada para innovar en productos financieros en las tendencias actuales, por ejemplo, en cloud banking. Dado que, por responsabilidad compartida, deben manejar unos estándares de seguridad entre las partes vinculantes cliente-proveedor (Amazon, 2024).

Para mitigar estos desfavorables hechos, las empresas contratan un grupo de desarrolladores, que dirigidos normalmente por un experto(interno) en el servicio, proyecto o api donde este alojado el código legado, dará las pautas del funcionamiento, los consumidores y las tecnologías con las que este se encuentre desplegado. Con dicha información, el grupo en cuestión hará un estimado de costo y tiempo para su implementación, tomando alrededor de 3 a 6 meses, sujeto a la complejidad y procesos internos de la organización, por ejemplo, comités, validaciones, pruebas, restricciones

de paso a producción, entre otros. Y al ser un proceso coyuntural en la entidad bancaria, no delegan esta responsabilidad en el equipo interno, dado que, normalmente están enfocados en sus respectivos factores de rendimiento, metas anuales de nuevos productos, etc., optando por contratar una empresa para tal fin, incrementando costos, dependencias de terceros, cláusulas, etc.

6.2. Formulación del problema

Conforme a lo descrito previamente, se propone la siguiente formulación del problema:

¿Cómo se pueden utilizar técnicas basadas en IA para mejorar los procesos de migración de código legado, garantizando un tiempo de implementación reducido en el sector bancario?

6.3. Sistematización del problema

- ¿Como determinar la eficiencia de los modelos de IA para la migración de código legado?
- ¿Cómo preservar la semántica del código legado durante la migración?
- ¿Cómo se pueden adaptar las técnicas del procesamiento del lenguaje natural para interpretar y transformar el código heredado de manera eficaz en la migración de código?
- ¿Qué métrica se debe aplicar para evaluar la eficacia de las herramientas de migración basadas en IA?

7. Marco teórico o conceptual

7.1 Marco Conceptual

7.1.1 Código legado

(Feathers, 2004) escribió: “El código heredado es código que obtuvimos de otra persona. Quizás nuestra empresa adquirió código de otra; quizás miembros del equipo original se trasladaron a otros proyectos. El código heredado es el código de otra persona.” En la misma línea, se ubica (Svensson, 2018) “Una definición recomendada de código heredado es código sin pruebas, con poca o ninguna documentación y, por lo general, difícil de entender, con o sin documentación. A partir de las observaciones realizadas durante la implementación de los prototipos, se formulan cinco pautas para ayudar a identificar y descubrir la evolución del código heredado.”

Entre las pautas que describe el autor sugiere (Svensson, 2018):

- “Falta de descripción de por qué un determinado bloque de código realiza una tarea.
- Documentación mal escrita o nula.
- Construido sobre un diseño de software antiguo o defectuoso.
- Dependencia de demasiadas bibliotecas y módulos externos.
- Sintaxis incorrecta.”

Para acotar sobre este tema (IBM, 2025) indica: “El código heredado se refiere al código de software que aún cumple su función, pero se desarrolló con tecnologías obsoletas. También incluye el código escrito con hardware o sistemas operativos obsoletos, compiladores o interfaces de programación de aplicaciones (API) obsoletos, o lenguajes de programación o entornos de desarrollo de software obsoletos. Como resultado, el código heredado ya no cumple con los nuevos estándares de codificación, los principios de diseño de software actuales ni la arquitectura informática más reciente.”

7.1.2 Sintaxis y semántica en código

Se refiere a esto términos, porque, serán evaluados como métricas más adelante. En el primer caso, la sintaxis en código refiere a las reglas que definen la estructura y formato de código en un lenguaje de programación, es decir, expresiones, oraciones y unidades de programa (variables); similar a la gramática en el lenguaje humano. Por ejemplo, el uso de la palabra “while”. (Sebesta, 2012)

Mientras, la semántica es el significado de esas expresiones, oraciones y unidades de programa, por ejemplo, empleando la anterior palabra “while” en la siguiente oración: `while(boolean_expr) statement`. (Sebesta, 2012). Enfocado en esta parte, chequear reglas semánticas de un lenguaje es la parte esencial en un compilador y aterrizado a esta propuesta, es un pilar para evaluar en la migración de código.

7.1.3 Large Language Model (LLM)

(Wayne Xin Zhao, 2025) parte explicando desde el modelo del lenguaje (LM), siendo este, uno de los mayores enfoques para el avance de la inteligencia lingüística de las máquinas. Donde LM busca modelar la probabilidad generativa de secuencias de palabras y puede ser dividido en cuatro escenarios de desarrollo, a saber: Modelos de lenguaje estático (SLM), modelo lenguaje neuronal

(NLM), modelo de lenguaje pre-entrenado (PLM) y, el que atañe en este proyecto, grandes modelos de lenguaje o más conocidos como LLM.

Típicamente, los LLM refiere a un modelo de lenguaje transformados que contiene miles de millones o más de parámetros, entrenados con datos de texto masivos, por ejemplo: GPT-4°, LLaMA, PaLM, entre otros. (Wayne Xin Zhao, 2025)

Resaltando de este modelo, contiene las siguientes habilidades emergentes:

- Aprendizaje en contexto.
- Seguimiento a la instrucción.
- Razonamiento paso a paso.

La introducción del LLM crea varias oportunidades en las tareas de ingeniería en software, entre ellas: pruebas, refactorización, detección de bugs, revisión de código y, recientemente, migración de código (Ziftci, et al., 2025)

7.1.4 Pruebas

Como se ha visto en puntos anteriores, una de los ítems importantes para validar la migración de código, consiste en las pruebas, por tal motivo, se hace necesario enfatizar en esta parte. En particular y de acuerdo con la literatura relacionada previamente, las más relevantes para este caso, son las pruebas unitarias e integrales.

Pruebas unitarias:

Enfocado en pruebas individuales según las especificaciones generadas durante el diseño del servicio. Los datos de prueba pueden generarse de forma que cubran todos los aspectos de la especificación o todas las características estructurales de la unidad (Mili & Tchier, 2015).

Pruebas integrales:

(Aniche, 2022) Es el nivel de prueba que integra el código del servicio con terceras partes. Esta ayuda a testear múltiples componentes de un sistema a la vez, enfocándose en las interacciones entre ellos, determinando su conexión y comportamiento exitoso.

7.1.5 Arquitectura limpia

Conforme al incremento que ha tenido la arquitectura limpia (Clean Architecture), aunque no es adoptado universalmente, sus principios han incrementado la influencia en el diseño del software moderno, donde se busca gestionar la complejidad, pero garantizando la mantenibilidad. Es por ello

que, en esta propuesta, la salida del código de migración más allá del lenguaje esperado, es la arquitectura con la cual se busca entregarla.

Esta arquitectura, tiene como base los principios solid, siendo estos (Isha & Vishalakshmi, 2023):

- Principio de responsabilidad única (SRP): Cada módulo o clase en el sistema solo tiene una tarea que completar.
- Principio Open-Closed (OCP): El Sistema debe cambiar mediante la adición de código y, no, desde la modificación del código existente.
- Principio de sustitución de Liskov (LSP): Los sistemas deben construirse con componentes intercambiables.
- Principio de segregación de interfaces (ISP): Un componente no puede depender de otro si no lo utiliza.
- Principio de inversión de dependencias (DIP): La ejecución de implementaciones de nivel inferior no puede afectar la aplicación de políticas de nivel superior.

Esta arquitectura cuenta con cuatro capas, descritas a continuación (Isha & Vishalakshmi, 2023):

- Entidades: O lógica de dominio, Incluye el propósito principal del servicio.
- Casos de uso: Incluye los casos de uso específicos del negocio, detalles sobre los clientes, además, el código reutilizable.
- Adaptadores de interfaz: O puerta de enlace. Se gestiona la transferencia de datos desde y hacia cada servicio. Aquí se realiza la manipulación de datos y las llamadas a las funciones principales.
- Frameworks y drivers: Incluye las interfaces a las que está expuesto el usuario. Incluye web y servidores. También, incluye tarea de soporte o trabajos automatizados.

7.2 Marco Teórico

7.2.1 Investigaciones previas

En estudios sobre el tema, expuestos por (De Marco, Iancu, & Asinofsky, 2018), indican una metodología para la migración de una aplicación en COBOL a Java y, a su vez, en un sistema operativo Linux, compuesta por los siguientes pasos:

- Coleccionar el inventario o código.
- Descomponer en paquetes de trabajo.

- Remodelar base de datos.
- Migración de datos.
- Conversión de Código JCL.
- Pruebas unitarias.
- Pruebas aceptación del usuario.
- Transición.

De este estudio, cabe recalcar que el éxito de la traslación de código y datos, radica en las pruebas y, donde, la aplicación es funcionalmente equivalente a la original.

Así mismo, otro estudio donde especifica una metodología para la migración de aplicaciones legadas (Hughes, et al., 1997), comenta los siguientes 10 pasos:

- Identificar los servicios y una migración de aplicación: Determinar la factibilidad de la migración del servicio, con base en, su costo/beneficio.
- Evaluar el impacto del entorno de implementación y la arquitectura objetivo.
- Establecer un entorno de desarrollo de ingeniería: Consiste identificar el entorno de desarrollo y las herramientas necesarias para la ingeniería.
- Establecer la línea base de la aplicación heredada: Indica el proceso de obtener, compilar, instalar, configurar y probar el código heredado
- Obtener un servicio reutilizable.
- Analizar el código heredado: Identificar el código heredado y determinar como modificarlo.
- Remodularizar el código heredado para invocar servicios reutilizables.
- Realizar pruebas de integración de la aplicación migrada: Constatar que el código migrado, cumple con la funcionalidad inicial.
- Documentar la aplicación migrada: Describir las definiciones de objetos, llamadas a servicios y el código rediseñado.

Otros estudios establecen (Barden Rubbo, Kessler Piveta, & Daltro, 2012) la inferencia de reglas para la migración de código, enfocado en el polimorfismo. Allí describe, con un soporte matemático en lógica y teoría de conjuntos, desde la definición de variables, nombre de métodos, expresiones, declaraciones y las restricciones.

Por otro lado, (Aleksyuk & Itsykson, 2017), describe una metodología para la migración de librerías, acá plantea cinco pasos, que consisten en:

- Extracción de trazas: Una traza contiene una lista de elementos de código colocado en el orden de activación.
- Mapeo de trazas: Incluye el seguimiento del programa y el mapeo al modelo de la librería de origen.
- Calcular la traza equivalente: Es uno de los pasos más importantes y es un proceso inmediato de búsqueda de un reemplazo para una traza del modelo.
- Mapeo de una nueva traza hacia los modos del árbol de sintaxis abstracta (AST).
- Transformación del programa: Un conjunto de nodos AST que se obtiene del paso anterior son reemplazadas en el árbol de sintaxis.

En este estudio, resalta nuevamente las pruebas como eje primordial para corroborar el éxito de la migración.

En estudios recientes, (Ishaani, Behrooz, & Anmol, 2024), donde se observa una incursión en el uso de LLM para la migración de código, empleando una herramienta para tal fin, siendo esta Amazon Q. Aquí buscan los autores, encontrar la relación entre la IA y el humano, destacando dos roles que desempeñan los segundos. Estos son: director y supervisor. En el primer caso, el humano puede ayudar a guiar la IA en el principio del código migrado para generar una correcta salida; en el segundo caso, el humano revisa edita y corrige la salida generada por la IA. También, discute la importancia de las pruebas para establecer una confianza en la migración. En este último aspecto, define tres escenarios: salida con alucinaciones, salida correcta pero incompleta y salida correcta.

En otros casos, (Alder, 2024), indica 4 fases para la migración empleando para tal fin la herramienta Sourcegraph AI, descritas a continuación:

- Evaluación inicial y planificación: La herramienta descrita, analiza repositorio con sus estructuras, dependencias y patrones de código.
- Definición de migración de reglas: Permite al usuario ingresar las reglas de migración a través de un lenguaje natural.
- Ejecución de migración: Esta a su vez, consiste en las transformaciones conscientes del contexto y detección de código sin buenas prácticas.
- Validación y testeo: Valida a partir de pruebas automatizadas la migración de código, empleando pruebas unitarias por modulo, función o servicio.

7.3 Marco espacial

7.3.1 Sector Bancario

(Hall, 2023) indica la importancia del sector bancario en USA, siendo uno de los sectores de mayor relevancia y segmento de la economía. Siendo de uno de los grandes servicios de la industria financiera. Esta incluye otros subsectores enfocados la gestión de activos, seguros, capital de riesgo y capital privado. La principal función económica del sector bancario son tomar depósitos y hacer préstamos.

En Colombia (Oglietti, 2024), se observa como este sector cuenta con un músculo financiero importante, creciendo entre el 2019 y el 2023 alrededor de \$285 billones de pesos, un 42%, similar a la tendencia mundial que en ese periodo, estuvo en el 44%. Considerando lo anterior, cumple un rol en la estabilidad financiera del país, por lo tanto, es necesario que sean reguladas y supervisadas, en este caso, este rol lo cumple la Superintendencia Financiera de Colombia y la constitución de 1991, en el capítulo “Planteamiento del Problema” se explica el impacto y la relación de estos entes en este sector.

7.3.2 Rol Inteligencia Artificial en el sector bancaria

(Adamyk, Benson, Adamyk, Fraczek, & Chinnaswamy, 2024) Evidencia como adoptar herramientas de IA en el sector bancario ha mejorado significativamente los servicios financieros para ofrecer alta eficiencia y nuevas capacidades, con las siguientes aplicaciones:

- Detección y prevención de fraude.
- Mejoramiento del servicio al cliente.
- Gestión de riesgo.
- Cumplimiento normativo y monitoreo.
- Servicios de banca y asesoramiento personalizados.
- Gestión de portafolio y análisis y crédito.

Se observa como la presente propuesta, toca varios de estos puntos, entre ellos: Gestión de riesgo y cumplimiento, donde, aumenta la precisión del cumplimiento y reduce la necesidad de grandes equipos de cumplimiento (Adamyk, Benson, Adamyk, Fraczek, & Chinnaswamy, 2024).

7.3.3 Sistemas de integración

Cuando se comenta sobre el desarrollo de software, normalmente son dos términos que retrotraen en la mente de cualquier programador, siendo estos: front-end (abarca todo lo visible y funcional de cara al cliente) y backend (contiene la lógica de negocio e interactúa con base de datos, autenticaciones, entre otros). Sin embargo, hay otro concepto menos recurrente que los anteriores, tratándose de los sistemas de integración y, tal, como lo plasma (Singireddy, 2025) las entidades financieras modernas operan cada vez más con sistemas más complejos y pueden gestionar de 500 a 1000 aplicaciones conforme al tamaño de la misma. Según (Singireddy, 2025), las empresas financieras destinan entre un 15% a 20% de sus presupuestos tecnológicos anuales a iniciativas de integración de sistemas, alcanzando una inversión de 4200 millones de dólares anuales referentes al 2024, destinadas a que sistemas sean robustos para garantizar la interoperabilidad entre la infraestructura existente y las tecnologías emergentes. Es acá, donde los sistemas de integración, también conocidos como capa de integración o en otros contextos como middleware, cumple un papel fundamental, por cuanto, son intermediarios facilitando la traducción de protocolos, transformación de formato de datos, unificación de estándares de comunicación, homologación de mensajería, entre sistemas que operan sobre bases tecnológicas diversas. A continuación, la Ilustración 1 muestra las tecnologías y metodologías de integración bancario.

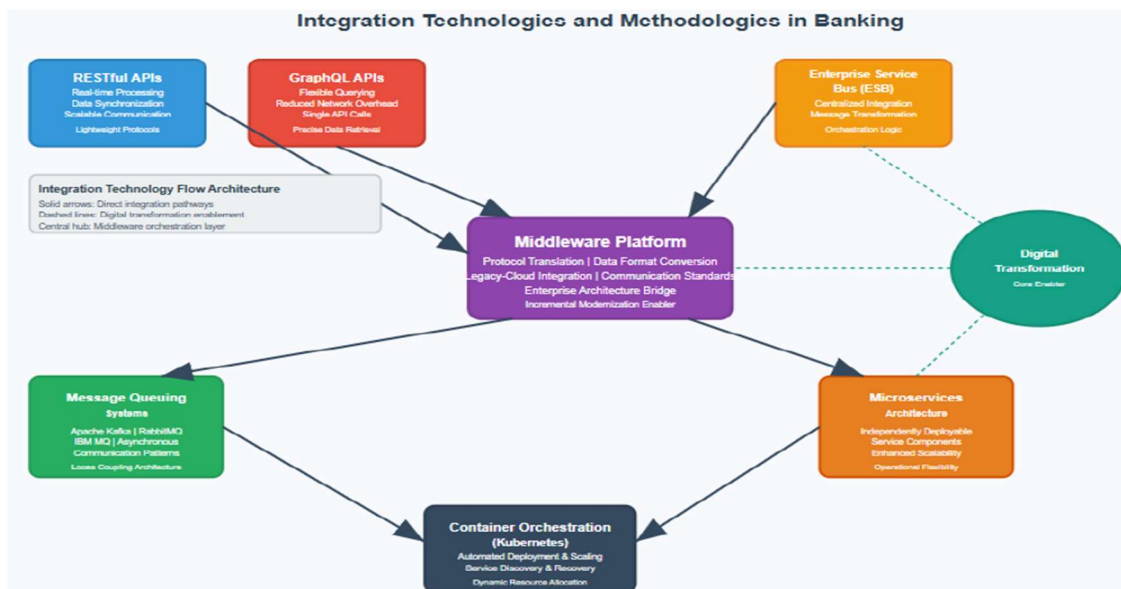


Ilustración 1 Tomada de System Integration in Financial Services: A Comprehensive Analysis of Modern Banking Architecture (Singireddy, 2025)

Las funciones que enmarcan la capa de integración se listan en la Tabla 1. (Singireddy, 2025), siendo uno de los campos de interés el que aplicación en la transformación de datos, que señala uno de los puntos importantes en esta tesis y hace referencia al código legado.

Tabla 1

Componentes de la tecnología de integración y métricas de rendimiento – adaptada de (Singireddy, 2025)

Componente Tecnológico	Requerimiento de implementación	Desafío operacional	Rendimiento estándar	Mandato de cumplimiento
API Autenticación	Systema basado en token	Administración protocolo Oauth	Optimización tiempo respuesta	Regulación de servicios financieros
Transformación de datos	Conversión de formato	Integración de arquitectura legada	Soporte de usuario concurrente	Preservación de la precisión de los datos
Frameworks de seguridad	Implementación de limitación de velocidad	Complejidad de validación de entrada	Estándares de disponibilidad del sistema	Requerimientos PCI DSS
Sistemas de rendimientos	Optimización de la latencia	Gestión de transacciones pico	Estándares capacidad de procesamiento	Cumplimiento regulatorio
Metodologías de prueba	Validación de escenario	Verificación multiinterfaz	Prueba de condición de carga	Cobertura integral
Estrategías de despliegues	Implementación incremental	Minimización disrupción operacional	Continuidad del servicio	Coordinación de partes interesadas

7.3.3.1 IBM Integration Bus versión 10 (IIB10) y App Connect Enterprise (ACE)

Una de las herramientas de integración usada con frecuencia en el ámbito bancario, debido a su robustez, soporte, entre otras características sobresalientes, empleada para facilitar la comunicación entre diferentes aplicaciones, soportando varios tipos de protocolos y formatos.

Ambos son productos desarrollados por IBM. El soporte de IIB10 se dio hasta el 30 de abril del 2022, dando una extensión de soporte por tres años, los cuales caducaron el año pasado (IBM, 2023). El producto evolucionó en la herramienta ACE, está se encuentra al presente en la versión 13.

7.3.4 Scaffold

En uno de los apartes comentados en 7.1.5, se mencionaba las partes comunes en una arquitectura limpia, las cuales son: adaptadores, entidades, casos de uso y drivers, donde, teniendo como base los principios SOLID, da un gran margen de posibilidades para concretar una arquitectura en un proyecto específico, respetando la separación de responsabilidades mediante capas definidas, centrándose en el dominio y las reglas de negocio (Bancolombia Tech, 2020). La Ilustración 2 enmarca lo comentado previamente:

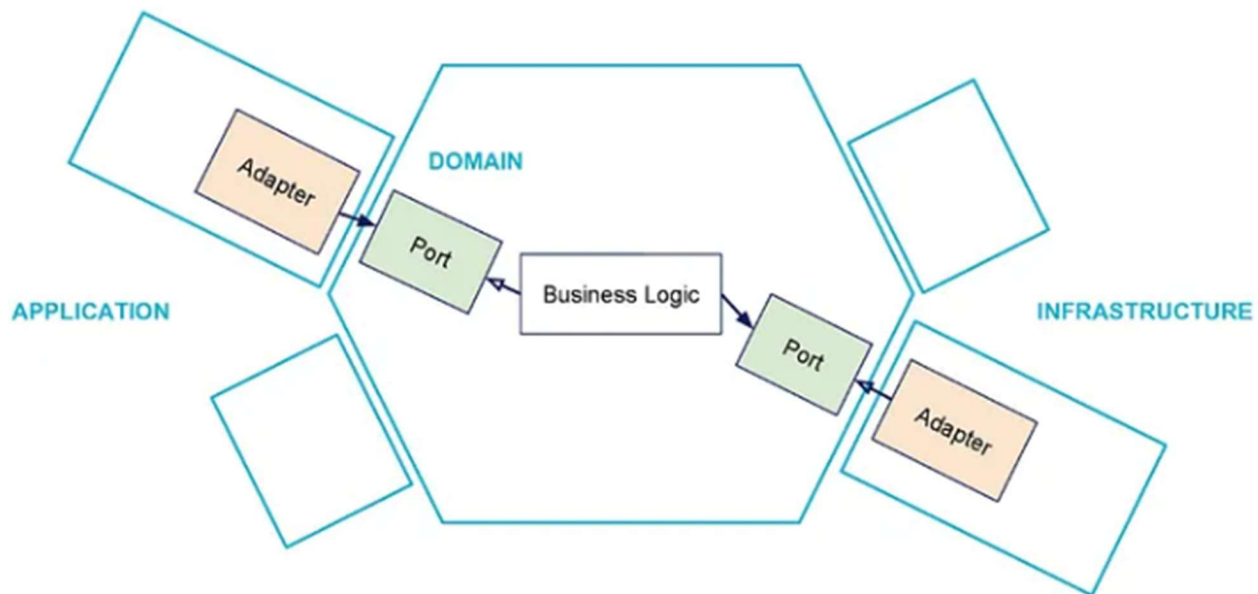


Ilustración 2 Ejemplo implementación de Clean Architecture – tomado de (Bancolombia Tech, 2020)

Es acá, donde una de las entidades bancarias más grandes de Colombia, buscando actualizar sus aplicaciones legadas y minimizar los costes de implementación y despliegue, opta por crear su propio plugin de clean architecture conocido como scaffold, soportado en código abierto a saber: un manejador de dependencias como gradle, un lenguaje de programación como java y su marco de trabajo por excelencia, spring boot. Con esto busca una forma de crear un proyecto de arquitectura limpia basándose en los principios de separación de responsabilidades y la regla de dependencia, donde debe apuntar desde las capas externas hacia las internas (Grupo Bancolombia, 2026). En la Ilustración 3 muestra la interacción entre dichas capas.

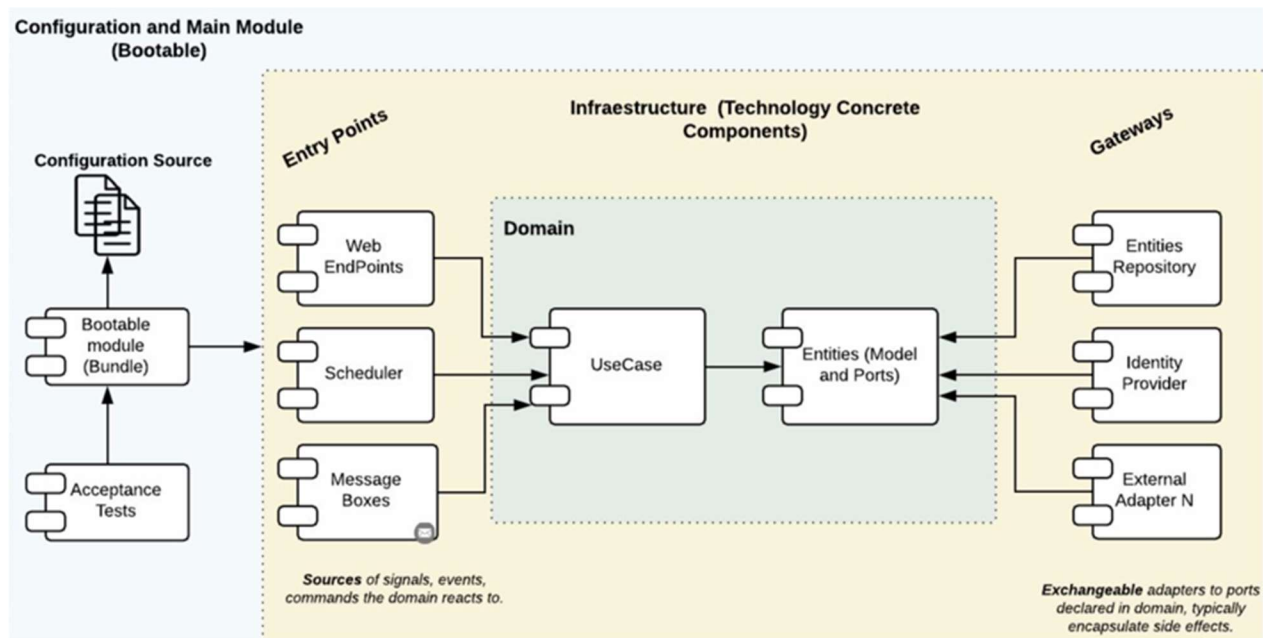


Ilustración 3 Implementación del plugin de Scaffold – tomado de (Grupo Bancolombia, 2026)

8. Objetivos

8.1 Objetivo general

Desarrollar una metodología basada en IA para la migración de código legado que reduzca el tiempo de implementación en el sector bancario.

8.2 Objetivos específicos

- Evaluar diferentes modelos de IA identificando el más eficiente para las tareas de migración de código.
- Formular un conjunto de reglas de IA para garantizar la preservación semántica durante la migración del código.
- Validar los procesos de migración basados en IA mediante la integración de técnicas avanzadas de procesamiento del lenguaje natural.
- Proponer una métrica estandarizada basada en IA para evaluar la calidad del código migrado.

9 HIPÓTESIS

Una metodología basada en IA, empleada en la migración de código legado, reduce significativamente el tiempo de implementación; comparándola con los enfoques tradicionales o manuales.

10 METODOLOGÍA

Como insumos en la migración de código legado se usará como punto de partida la tecnología de integración descrita en aparte 7.3.3.1, esta es IIB10 y se llevara a una arquitectura limpia implementada en el plugin Scaffold (aparte 7.3.4). Para ello lo primero es definir los prompts que permitirán extraer la información de ambas tecnologías, donde se hará una extracción desde las páginas web y otra desde los repositorios de los tutoriales que vienen en la herramienta IIB10.

De igual forma, como punto de partida se empleará el modelo ChatGpt 5.5 en su suscripción PRO. Lo anterior, con base en el más inteligente a la fecha de este documento y respaldado por (Analysis, 2026).

10.1 Creación de prompts código legado

Para arrancar con el proceso, se optó inicialmente obtener una base de datos que no afectara la producción actual de la entidad bancaria. Para lo anterior, se buscaron los ejemplos(tutoriales) que viene con la herramienta IIB10. Contando con 65 proyectos listados a continuación (Tabla 2):

Tabla 2
Insumo base de datos apartir de los proyectos base en la herramienta IIB10

ÍTEM	NOMBRE	ÍTEM	NOMBRE
1	AggregationCallableAsync	33	ISO8583
2	AggregationCallableAsyncBackend1	34	KafkaConsumerApplication
3	AggregationCallableAsyncBackend2	35	KafkaProducerApplication
4	AggregationHTTPAsync	36	LargeMessages
5	AggregationHTTPAsyncLib	37	LargeMessagesJava
6	AggregationHTTPBackend1	38	LogMessages
7	AggregationHTTPBackend2	39	LookupTableCSVdata

8	AggregationMQ	40	LookupTable_UsingMappingGlobalCache
9	AggregationMQBackend1	41	LoopBack
10	AggregationMQBackend2	42	MQTTSubscribe
11	AggregationRESTAsync	43	PrimeNumbers
12	AggregationRESTBackend1	44	Resequence
13	AggregationRESTBackend2	45	RESTAsyncRequest_API
14	AggregationSOAPAsync	46	RESTAsyncRequest_Client
15	AggregationSOAPAsyncLib	47	RESTAsyncRequest_SharedLibrary
16	AggregationSOAPBackend1	48	RESTRequest_API
17	AggregationSOAPBackend2	49	RESTRequest_Client
18	BARfiles	50	RESTRequest_SharedLibrary
19	BTM_RETAIL_APP	51	Salesforce_Retrieve
20	BTM_RETAIL_SHARED_LIBRARY	52	Salesforce_Schemas
21	CallableFlowBARFiles	53	Sequence
22	CallableTimestamp	54	TCPIPClient
23	CallableTimestampAlternate	55	TCPIPServer
24	CallingParentApplication	56	TemperatureConverter
25	COBOL-Purchases	57	TLOG-ACE
26	CommaSeparatedValues	58	Transformation_DotNet
27	CustomerDatabaseJava	59	Transformation_DotNet_DotNET
28	CustomerDatabaseV1	60	Transformation_ESQL
29	DataLibrary	61	Transformation_Java
30	EDIFACT-Common	62	Transformation_JavaJava
31	EDIFACT-D03B	63	Transformation_Map
32	HL7-2.7	64	Transformation_XSL
33	HTTPInputApplication	65	WorkloadManagementApplication

Cada proyecto tiene la siguiente estructura: una parte grafica con la extensión msgflow, otra, donde se codifica. Esta segunda es una extensión del SQL (Structured Query Language) bastante empleado en el manejo de bases de datos relacionales, conocido en este contexto como ESQL. Normalmente un proyecto en IIB10 está estructurado de la siguiente forma (Ilustración 4):

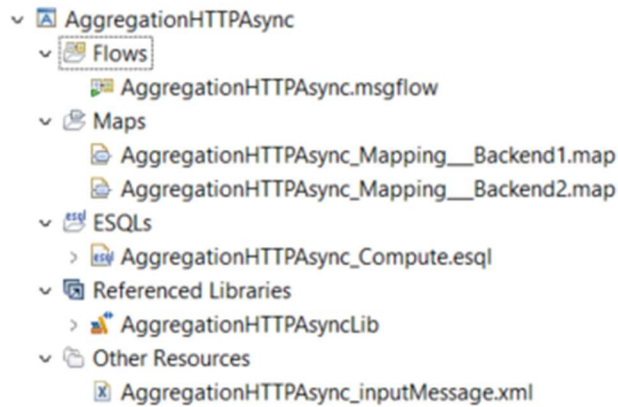


Ilustración 4 Estructura proyecto en IIB10

- Flujos y subflujos (Ilustración 5): contiene la parte gráfica (ver Ilustración 4), en el que, se dispone de diferentes nodos (cajas o recuadros) con especificaciones particulares, a saber: entradas y salidas (a MQ, REST, SOAP, etc.), transformación de mensajería, envío de correos, conexión a base de datos, temporizadores, entre otros.

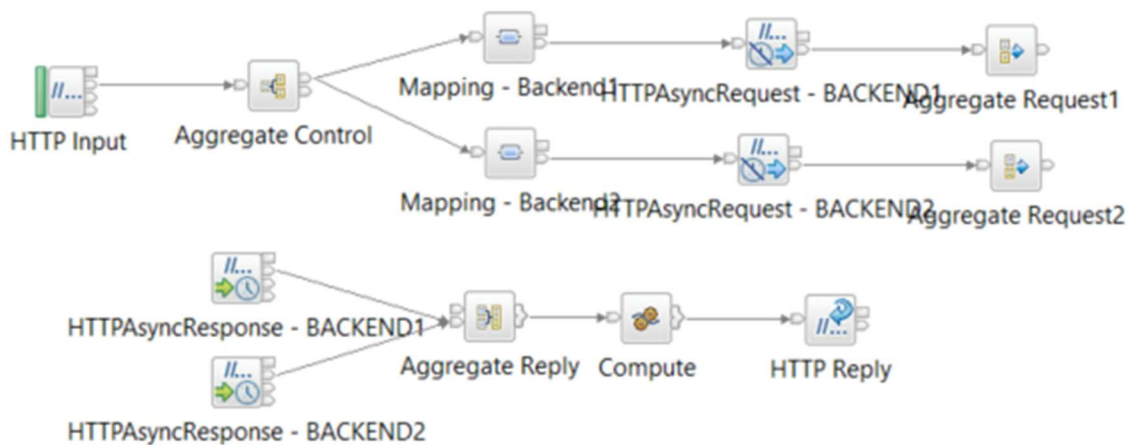


Ilustración 5. Ejemplo parte gráfica - proyecto reserva tutorial

Cada nodo o recuadro tiene a su vez sus propias características, que serán clave evaluarlas para la migración. Una muestra de lo comentado (Ilustración 6), es el nodo se resalta el nodo “Http Asynchronous Response”, allí en el recuadro inferior, se observan características como tipo de mensaje, tipo de validación, monitoreo, etc.

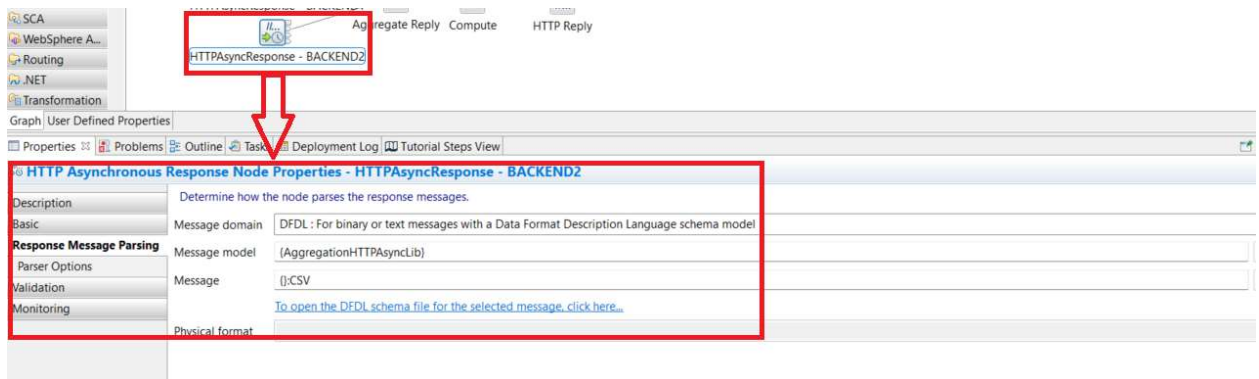


Ilustración 6. Detalle ejemplo de propiedades nodo “Asynchronous Response”

Continuando con la parte gráfica, estos archivos tienen la extensión msgflow, sin embargo, su base es en lenguaje Extensible Markup Language (XML), esta aclaración, facilita la manipulación e iteración con los LLM. Esto se observa en la Ilustración 7, donde se tiene una muestra del nodo resaltado en la ilustración previa. Para matizar, hay dos partes cruciales que serán fundamentales en la migración de código, estos son: “nodes” y “connection”, el primero, contiene la información intrínseca del nodo, mientras, el segundo, muestra como interactúa el nodo con su entorno.

```

<?xml version="1.0" encoding="utf-8" ?>
<ecore:EPackage xmlns:ecore="http://www.eclipse.org/emf/2002/Ecore" xmlns:xmi="http://www.omg.org/XMI" xmlns:ComIbmLabel.msgnode="ComIbmLabel.msgnode"
xmlns:ComIbmRouteToLabel.msgnode="ComIbmRouteToLabel.msgnode" xmlns:ComIbmInput.msgnode="ComIbmInput.msgnode" xmlns:ComIbmReply.
msgnode="ComIbmReply.msgnode" xmlns:ecore="http://www.eclipse.org/emf/2002/Ecore" xmlns:eflow="http://www.ibm.com/ubi/2005/eflow"
xmlns:getAggregatedResponse.subflow="getAggregatedResponse.subflow" xmlns:utility="http://www.ibm.com/ubi/2005/eflow_utility"
nsURI="gen/AggregationCallableAsync.msgflow" nsPrefix="gen_AggregationCallableAsync.msgflow">
<eclassifiers xmi:type="eflow:FCMComposite" name="FCMComposite_1">
<eSuperTypes href="http://www.ibm.com/ubi/2005/eflow#/FCMBlock"/>
<translation xmi:type="utility:TranslatableString" key="AggregationCallableAsync" bundleName="gen/AggregationCallableAsync"
pluginId="AggregationCallableAsync"/>
<colorGraphic16 xmi:type="utility:GIFFileGraphic" resourceName="platform:/plugin/AggregationCallableAsync/icons/full/obj16/
AggregationCallableAsync.gif"/>
<colorGraphic32 xmi:type="utility:GIFFileGraphic" resourceName="platform:/plugin/AggregationCallableAsync/icons/full/obj30/
AggregationCallableAsync.gif"/>
<composition>
<nodes xmi:type="ComIbmInput.msgnode:FCMComposite_1" xmi:id="FCMComposite_1_1" location="25,150" URLSpecifier="/"
aggregationCallableAsync/v1" parseQueryString="true" messageDomainProperty="JSON" faultFormat="HTML">
<translation xmi:type="utility:ConstantString" string="HTTP Input"/>
</nodes>
<nodes xmi:type="ComIbmRouteToLabel.msgnode:FCMComposite_1" xmi:id="FCMComposite_1_2" location="139,146">
<translation xmi:type="utility:ConstantString" string="Route To Label"/>
</nodes>
<nodes xmi:type="ComIbmReply.msgnode:FCMComposite_1" xmi:id="FCMComposite_1_3" location="525,150">
<translation xmi:type="utility:ConstantString" string="HTTP Reply"/>
</nodes>
<nodes xmi:type="ComIbmLabel.msgnode:FCMComposite_1" xmi:id="FCMComposite_1_4" location="125,225" labelName="getAggregatedResponse"
<translation xmi:type="utility:ConstantString" string="getAggregatedResponse (Label)"/>
</nodes>
<nodes xmi:type="getAggregatedResponse.subflow:FCMComposite_1" xmi:id="FCMComposite_1_5" location="325,225">
<translation xmi:type="utility:ConstantString" string="getAggregatedResponse (Implementation)"/>
</nodes>
</composition>
<connections xmi:type="eflow:FCMConnection" xmi:id="FCMConnection_1" targetNode="FCMComposite_1_4" sourceNode="FCMComposite_1_1"
sourceTerminalName="OutTerminal.out" targetTerminalName="InTerminal.in"/>
<connections xmi:type="eflow:FCMConnection" xmi:id="FCMConnection_2" targetNode="FCMComposite_1_5" sourceNode="FCMComposite_1_4"
sourceTerminalName="OutTerminal.out" targetTerminalName="InTerminal.Input_1"/>
<connections xmi:type="eflow:FCMConnection" xmi:id="FCMConnection_3" targetNode="FCMComposite_1_3" sourceNode="FCMComposite_1_5"
sourceTerminalName="OutTerminal.Output_1" targetTerminalName="InTerminal.in"/>
</connections>
<propertyOrganizer>
<stickyBoard>
<stickyNote location="25,25">
<body xmi:type="utility:ConstantString" string="This message flow is automatically generated.&#xA;Do not edit this message flow.&
#xA;If you edit the message flow, your changes will be overwritten without a warning.&#xA;"/>
</stickyNote>
</stickyBoard>
</propertyOrganizer>
</eclassifiers>
</ecore:EPackage>

```

Ilustración 7. XML flujo gráfico referido sobre la Ilustración 6.

Con lo comentado previamente, se comienza a definir las reglas para extraer los campos relevantes y definir la base de datos relacionales de las propiedades de los nodos. Primero, se usa un archivo como laboratorio antes de evaluar todos los archivos msgflow, con el fin de, afinar la primera regla de extracción de información para el migrado de código. Esto se logra después de tres iteraciones, debido a que, estaba excluyendo propiedades importantes y en el objeto nodo existe unos objetos internos, por lo que, fue necesario especificar en cada iteración, para mejorar la regla inicial. Cabe aclarar que este tipo de archivo tiene como base en lenguaje esto facilita la iteración desde los LLM, con esta anotación, se propone el PROMPT 1 de inspección:

PROMPT 1

Parse the file AggregationCallableAsync.xml to extract all properties from both the nodes and connections objects. For each node, if a translations element exists, decompose its sub-properties by splitting keys on the period (.) delimiter, and append these separated translation properties as individual columns in the output table.

Como resultado, se obtiene la Tabla 3:

Tabla 3
Propiedades objeto "node"

PROPIEDADES	OBJETO 1	OBJETO 2
xmi:id	FCMComposite_1_1	FCMComposite_1_2
xmi:type	ComIbmWSInput.msgnode: FCMComposite_1	ComIbmRouteToLabel.msgnode: FCMComposite_1
location	25,150	139,146
URLSpecifier	/aggregationcallableasync/v1*	—
parseQueryString	TRUE	—
messageDomainProperty	JSON	—
faultFormat	HTML	—
labelName	—	—
translation.xmi:type	utility:ConstantString	utility:ConstantString
translation.string	HTTP Input	Route To Label

Para el objeto “connection”, se indica las propiedades extraídas en la

Tabla 4
Propiedades objeto "connection"

PROPIEDAD	OBJETO 1	OBJETO 2	OBJETO 3
xmi:id	FCMConnection_1	FCMConnection_2	FCMConnection_3
sourceNode	FCMComposite_1_1	FCMComposite_1_4	FCMComposite_1_5
sourceTerminalName	OutTerminal.out	OutTerminal.out	OutTerminal.Output_1
targetNode	FCMComposite_1_2	FCMComposite_1_5	FCMComposite_1_3
targetTerminalName	InTerminal.in	InTerminal.Input_1	InTerminal.in

Con esta información, se puede observar:

- Se contempla que existe una relación entre las dos tablas con la propiedad xmi:id.
- Hay propiedades, que no aportan a la migración de código generando ruido, entre ellos:
- Objeto node: location, translation.xmi:type.
- Con lo anterior, se crea el siguiente PROMPT 2:

PROMPT 2

Remove the properties location, and translate.xmi:type from the node table.

También, es posible crear, a partir de estas tablas, un modelo entidad relación (MER) resultando en la Ilustración 8.

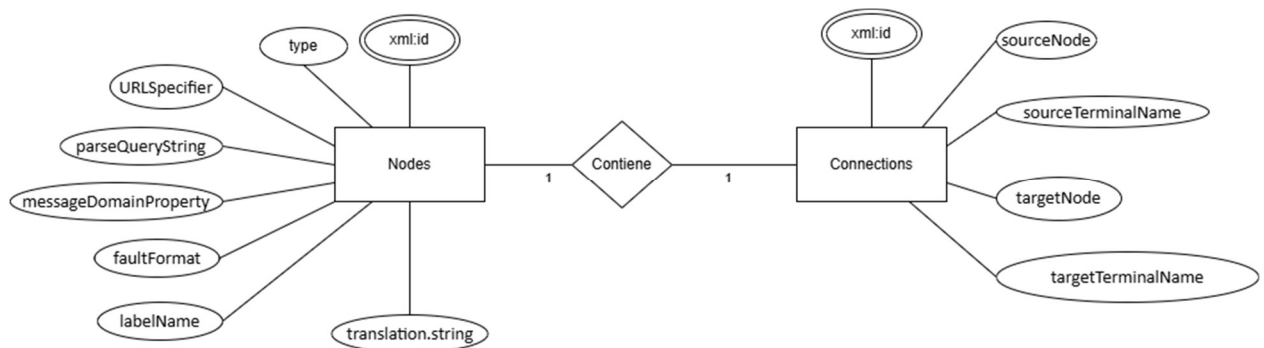


Ilustración 8. MER tablas "nodes" y "connections"

Con esta información, se procede a extraer la información de los 65 proyectos establecidos en la Tabla 2, para obtener lo descrito en las Tabla 3 y Tabla 4. Inicialmente la licencia pro de ChatGPT permite permite subir 25 archivos por proyecto (diferenciados por carpetas), lo que implica subir cada lote en dos ocasiones.

La regla mejorada, con base en la descrita con en el PROMPT 4 es el siguiente:

PROMPT 3 – parte 1

Choose the 25 files that are uploaded in this folder and extract all properties from the nodes objects. For each node, if a translations object is present, separate its properties, only translations, by the period (.) in their names and include these as additional columns in the resulting table. Organize the table, leaving the following columns at the top: source_file, xmi:id, xmi:type, translation.string. Remove the columns: location and translation.xmi:type, and arrange the remaining columns alphabetically. Save the result in a CSV file.

Ahora enfocada sobre el objeto “connections”, plasmada en la parte 2 del PROMPT 3:

Choose the 25 files that are uploaded in this folder and extract all properties from the connections objects. Avoid placing the name "connections" at the beginning of each property. Organize the table, leaving the following column at the top: source_file. Remove the property xmi:type and arrange the remaining columns alphabetically. Save the result in a CSV file.

Las propiedades condensadas para el objeto “node” son:

source_file, xmi:id, xmi:type, translation.string, URLSpecifier, acks, additionalInstances, aggregateName, allowQueryWSDL, asyncRequestCorrelator, asyncResponseCorrelator, callableInputEndpoint, componentLevel, computeExpression, connectionTimeout, connectorName, copyArrays, copyMessage, datasourceName, dbErrorHandling, dbQueryMode, dbType, decisionAttributeName, definitionType, description, dynamicApplicationName, dynamicEndpointName, EISRequestType, EISServerName, EISTimeout, enableRetry, encoding, endOfSequence, filePath, fileSize, flowOrder, groupName, httpMethod, httpProxyLocation, id, initialCount, isDefault, jdbcProviderName, logFile, maxErrors, maxInstances, maxRecords, messageDomain, messageFormat, messageModel, modifyOutputStream, MQMD_Write, MQMD_Write_Error, name, nodeClassName, operationName, outputType, override,

overrideConnection, overrideOperation, overrideReplyProtocol, overrideRequestProtocol, parameterName, parseTiming, parserName, pathToStoreSequenceNo, pattern, priority, processingMode, protocol, recordDetection, recordIsComplete, recordsPerMessage, reparse, retryInterval, retryThreshold, routeToLabel, SAPFunctionName, SAPRequestType, SAPRetryHandling, SAPServer, sequenceNumber, sequencePath, serviceName, shortDescription, source, SQLStatement, startOfSequence, stylesheetName, targetApplication, targetEndpointName, timeout, transactionMode, transactional, treatWarningsAsErrors, type, url, useDatabase, validateMaster, validateSequence, wsdlFile, xsltVersion.

Del objeto “connections”, las propiedades involucradas, son:

source_file, sourceNode, sourceTerminalName, targetNode, targetTerminalName y xmi:id.

- ESQL (Ilustración 9): Contiene la parte lógica y es donde se vinculan los nodos y sus diferentes partes para poderlos manipular. Acá es donde normalmente se ubica la lógica de negocio, aplicando condicionales, bucles, inicialización de variables, conversiones de tipo de dato, etc. Se puede crear librerías y funciones con retorno de dato o vacío.

```
CREATE COMPUTE MODULE AggregationHTTPAsync_Compute
CREATE FUNCTION Main() RETURNS BOOLEAN
BEGIN

    SET OutputRoot.Properties = InputRoot.ComIbmAggregateReplyBody.Backend1.Properties;
    DECLARE Backend1Ref REFERENCE TO InputRoot.ComIbmAggregateReplyBody.Backend1.DFDL.CSV.record;
    DECLARE Backend2Ref REFERENCE TO InputRoot.ComIbmAggregateReplyBody.Backend2.DFDL.CSV.record;
    CREATE LASTCHILD OF OutputRoot DOMAIN('XMLNSC');
    SET OutputRoot.XMLNSC.Message.Backend1.FieldA = Backend1Ref.field1;
    SET OutputRoot.XMLNSC.Message.Backend1.FieldB = Backend1Ref.field2;
    SET OutputRoot.XMLNSC.Message.Backend1.FieldC = Backend1Ref.field3;
    SET OutputRoot.XMLNSC.Message.Backend2.FieldA = Backend2Ref.field1;
    SET OutputRoot.XMLNSC.Message.Backend2.FieldB = Backend2Ref.field2;
    SET OutputRoot.XMLNSC.Message.Backend2.FieldC = Backend2Ref.field3;

    RETURN TRUE;
END;

END MODULE;
```

Ilustración 9. Código ejemplo en ESQL

- Librerías: Se ubican aquellas componentes, como subflujos, archivos ESQL o también es posible trabajar con archivos JAVA. Con la finalidad de reutilizar funciones o bloques de código en servicios transversales.

- Maps: Son archivos que contienen el mapeo de mensaje de entrada y salida.
- Otros: Puede contener archivos xml, json, tramas de texto plano, etc. de mensajes de entrada

Con esta estructura, el primer paso es migrar la parte gráfica, definiendo todos los posibles nodos que contiene la herramienta y, a su vez, clasificarlos, esto último es significativo, dado que, nos dará las pautas para la homologación con el patrón scaffold en clean architecture.

El PROMPT 4 extrae la información desde la documentación pública en el website de IBM:

PROMPT 4

List all the nodes in a table, using the following link: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=development-built-in-nodes>. Follow the link for each node and provide a summary of its functionality. Additionally, for each node, identify its type—specifically, whether it is an input node, an output node, etc. With answer generates as excel.

El resultado queda plasmado en la siguiente Tabla 5:

Tabla 5
Clasificación nodos de la herramienta IIB10

Node	Type	Summary
TimeoutControl	Control	Sets timeout
MQInput	Input	Starts flow by reading messages from MQ queue
MQGet	Input	Reads message without triggering flow
MQTTSubscribe	Input	Receives messages from MQTT topics
JMSInput	Input	Starts flow from JMS destination
JMSReceive	Input	Pulls JMS message synchronously
KafkaConsumer	Input	Consumes messages from Kafka
HTTPInput	Input	Receives HTTP requests
HTTPAsyncResponse	Input	Handles async HTTP responses
RESTAsyncResponse	Input	Handles async REST responses
SOAPInput	Input	Receives SOAP requests
SOAPAsyncResponse	Input	Handles async SOAP responses
TimeoutNotification	Input	Triggers timeout event
FileInput	Input	Reads files
EmailInput	Input	Receives email
DatabaseInput	Input	Reads from database

CallableFlowAsyncResponse	Input	Handles async response
CallableInput	Input	Entry point for callable flow
MQOutput	Output	Sends message to MQ queue
MQTTPublish	Output	Publishes messages to MQTT topics
JMSOutput	Output	Sends message to JMS
KafkaProducer	Output	Sends messages to Kafka
FileOutput	Output	Writes files
EmailOutput	Output	Sends email
MQOptimizedFlow	Processing	Optimizes MQ message handling
RegistryLookup	Processing	Retrieves service metadata
EndpointLookup	Processing	Resolves dynamic endpoints
Trace	Processing	Logs message content
TryCatch	Processing	Handles exceptions
Throw	Processing	Throws exception
Validate	Processing	Validates message
Database	Processing	Executes DB operations
MQReply	Reply	Sends reply using MQReplyTo
JMSReply	Reply	Sends reply to JMSReplyTo
HTTPReply	Reply	Sends HTTP responses
SOAPReply	Reply	Sends SOAP responses
CallableReply	Reply	Sends response
HTTPRequest	Request	Calls external HTTP service
HTTPAsyncRequest	Request	Sends async HTTP call
RESTRequest	Request	Calls REST service
RESTAsyncRequest	Request	Async REST call
SOAPRequest	Request	Calls SOAP services
SOAPAsyncRequest	Request	Async SOAP call
CallableFlowInvoke	Request	Calls another flow
CallableFlowAsyncInvoke	Request	Async flow call
Filter	Routing	Routes based on condition
Route	Routing	Routes message dynamically
RouteToLabel	Routing	Routes to label
Label	Routing	Defines destination label
FlowOrder	Routing	Controls execution order
MQHeader	Transformation	Manipulates MQ headers
JMSHeader	Transformation	Modifies JMS headers
JMSMQTransform	Transformation	Converts JMS to MQ
MQJMSTransform	Transformation	Converts MQ to JMS
HTTPHeader	Transformation	Manipulates HTTP headers
SOAPEnvelope	Transformation	Builds SOAP envelope
SOAPExtract	Transformation	Extracts SOAP body
Compute	Transformation	Uses ESQL to modify messages
JavaCompute	Transformation	Uses Java logic

Mapping	Transformation	Graphical data mapping
XSLTransform	Transformation	XML transformation using XSLT

El siguiente prompt es un conjunto de ellos (PROMPT 5 - PROMPT 20), donde se vincula la homologación entre el lenguaje obsoleto y al que se migrara, siendo estos: ESQL y JAVA, respectivamente.

Tipos de variables:

PROMPT 5

- ***Extract the data types from the following URL and its associated links. Organize the information into a table with examples. <https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=reference-esql-data-types-in-message-flows>. Create the result in an Excel file for download.***

Referencia de campos:

PROMPT 6

- ***Extract “ESQL field reference overview” the following URL and its associated links. In particular, on the following topics: “Namespace”, “Index”, “Type”, “Type constraints”, “Field references summary”, “Target field references” and “The effect of setting a field to NULL”. Organize the information into a table with examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=reference-esql-field-overview>. Create the result in an Excel file for download.***

Operadores:

PROMPT 7

- ***Extract the data operators from the following URL and its associated links. Organize the information into a table with example: <https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=reference-esql-operators>. Create the result in an Excel file for download.***

Basic Statements:

PROMPT 8

- *Extract the ESQL statements, particularly the section "basic statements", from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=reference-esql-statements>. Organize the information into a table and create the result in an Excel file for download.*

Message tree manipulation statements:

PROMPT 9

- *Extract the ESQL statements, particularly the section "Message tree manipulation statements", from the following URL. Supplement this with related links and also extract examples. <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=reference-esql-statements>. Organize the information into a table and create the result in an Excel file for download.*

Database update statements:

PROMPT 10

- *Extract the ESQL statements, particularly the section "Database update statements" from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=reference-esql-statements>. Organize the information into a table and generate the result in an Excel file for download.*

Node interaction statements:

PROMPT 11

- *Extract the ESQL statements, particularly the section "Node interaction statements" from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=statements-propagate-statement>, in particular with: "TO TERMINAL", "TO LABEL", "FINALIZE" and "DELETE". Organize the information into a table and generate the result in an Excel file for download.*

Other statements:

PROMPT 12

- *Extract the ESQL statements, particularly the section "Other statements" from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=reference-esql-statements>. Organize the information into a table and generate the result in an Excel file for download.*

Funciones:

PROMPT 13

1. Extract the “ESQL database state functions” from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-esql-database-state>. Organize the information into a table and generate the result in an Excel file for download.

PROMPT 14

2. Extract the “ESQL datetime functions” from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-esql-datetime>. Organize the information into a table and generate the result in an Excel file for download.

PROMPT 15

3. Extract the “ESQL numeric functions” from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-esql-datetime>. Organize the information into a table and generate the result in an Excel file for download.

PROMPT 16

4. Extract the “ESQL string manipulation functions” from the following URL. <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-esql-string-manipulation>. Supplement this with related links and also extract examples. Include the next function: CONTAINS function, ENDSWITH function, LEFT function, LENGTH function, LOWER and LCASE functions, LTRIM function, OVERLAY function, POSITION function, REPLACE function, REPLICATE function, RIGHT function, RTRIM function, SPACE function, STARTSWITH function, SUBSTRING function, TRANSLATE function, TRIM function, UPPER and UCASE functions, URLDECODE function, URLENCODE. Organize the information into a table and generate the result in an Excel file for download.

PROMPT 17

5. Extract the “ESQL field functions” from the following URL. Supplement this with related links and also extract examples: <https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-esql-field>. Organize the information into a table and generate the result in an Excel file for download.

PROMPT 18

6. **Extract the “ESQL list functions” from the following URL. Supplement this with related links and also extract examples, focus in CARDINALITY function, EXISTS function, SINGULAR function and THE function:**
<https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-esql-list>.
Organize the information into a table and generate the result in an Excel file for download.

PROMPT 19

7. **Extract the “Complex ESQL functions” from the following URL. Supplement this with related links and also extract examples:**
<https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-complex-esql>. **Focus on the following items: CASE function, CAST function, SELECT function, ROW constructor function, LIST constructor function, ROW and LIST combined, ROW and LIST comparisons, Supported casts, Implicit casts, Implicit CASTs for comparisons, Implicit CASTs for arithmetic operations, Implicit CASTs for assignment, Data types of values from external databases. Organize the information into a table and generate the result in an Excel file for download.**

PROMPT 20

8. **Extract the “Miscellaneous ESQL functions” from the following URL. Supplement this with related links and also extract examples:**
<https://www.ibm.com/docs/en/integration-bus/10.1.0?topic=functions-miscellaneous-esql>. **Focus on the following items: BASE64DECODE function, BASE64ENCODE function, CHANGEIDENTIFIERTIMEOUT function, COALESCE function, EVAL function, INSTANCESTOPPING function, NULLIF function, PASSTHRU function, SLEEP function, UUIDASBLOB function, UUIDASCHAR function. Organize the information into a table and generate the result in an Excel file for download.**

Con esta información recabada y almacenada en los archivos Excel, se procede a buscar su homologación en java, en su última versión, con el siguiente prompt:

PROMPT 21

Group all the files and create their equivalent in Java in its latest version with examples based on column ExampleESQL. Organize the information into a table and generate the result in an Excel file for download.

Aunque se crea el cuadro y una parte de la homologación, no específica del todo una conversión exitosa, por lo cual, se extrae exclusivamente la parte de los ejemplos y se concreta que enfatice la migración de ESQL a JAVA en estos casos, asociado con la Tabla 6:

Tabla 6
Homologación ESQL a JAVA

Category	ExampleESQL	JavaEquivalent	JavaExample
BOOLEAN	DECLARE v BOOLEAN; SET v = TRUE;	boolean / Boolean	boolean v = true;
DATE	DECLARE v DATE; SET v = DATE '2024-01-01';	java.time.LocalDate	java.time.LocalDate v = java.time.LocalDate.of(2024, 1, 1);
TIME	DECLARE v TIME; SET v = TIME '12:00:00.000000';	java.time.LocalTime	java.time.LocalTime v = java.time.LocalTime.of(12, 0, 0, 0);
GMTTIME	DECLARE v GMTTIME; SET v = GMTTIME '12:00:00';	java.time.OffsetTime (UTC)	java.time.OffsetTime v = java.time.OffsetTime.of(12, 0, 0, 0, java.time.ZoneOffset.UTC);
TIMESTAMP	DECLARE v TIMESTAMP; SET v = CURRENT_TIMESTAMP;	java.time.LocalDateTime	java.time.LocalDateTime v = java.time.LocalDateTime.now();
GMTTIMESTAMP	DECLARE v GMTTIMESTAMP; SET v = GMTTIMESTAMP '2024-01-01 12:00:00.000000';	java.time.OffsetDateTime (UTC)	java.time.OffsetDateTime v = java.time.OffsetDateTime.of(2024, 1, 1, 12, 0, 0, 0, java.time.ZoneOffset.UTC);
INTERVAL	DECLARE v INTERVAL DAY TO SECOND; SET v = INTERVAL '0 01:00:00';	java.time.Duration	java.time.Duration v = java.time.Duration.ofHours(1);
INTEGER (null)	DECLARE v INTEGER; SET v = NULL;	Integer	Integer v = null;
INTEGER	DECLARE v INTEGER; SET v = 123;	int / Integer	int v = 123;
DECIMAL	DECLARE v DECIMAL(10,2); SET v = 123.45;	java.math.BigDecimal	java.math.BigDecimal v = new java.math.BigDecimal("123.45");
FLOAT	DECLARE v FLOAT; SET v = 1.23E-4;	double	double v = 1.23e-4;
REFERENCE	DECLARE ref REFERENCE TO InputRoot.MQMD;	Node / JsonNode reference	org.w3c.dom.Node ref = mqmdNode; // or JsonNode ref = root.path("MQMD");
ROW	DECLARE r ROW; SET r.field='abc';	Map<String, Object> or POJO	java.util.Map<String, Object> r = new java.util.HashMap<>(); r.put("field", "abc");
BIT	DECLARE v BIT; SET v = B'0101';	byte (bit pattern)	byte v = 0b0101;
BLOB	DECLARE v BLOB; SET v = X'01234F';	byte[]	byte[] v = new byte[] { 0x01, 0x23, 0x4F };
CHARACTER	DECLARE v CHARACTER; SET v = 'hello';	String	String v = "hello";
XML Namespace write	SET OutputRoot.{http://example.org/myNS}Msg.Data.Value = 'X';	DOM element with namespace	org.w3c.dom.Element valueEl = doc.createElementNS("http://example.org/myNS", "Msg"); valueEl.setTextContent("X");
XML index	SET OutputRoot.XMLNSC.Orders.Order[2].Item = '123';	List index	orders.get(1).setItem("123");
Cast to INTEGER	SET v = (INTEGER)InputRoot.XMLNSC.Total;	Integer.parseInt	int v = Integer.parseInt(root.path("Total").asText());
Field set	SET OutputRoot.XMLNS.Msg.Data.Name = 'A';	Setter	output.getMsg().getData().setName("A");
Field null	SET OutputRoot.XMLNS.Msg.Data.Name = NULL;	Setter to null	output.getMsg().getData().setName(null);
Equal	IF a = b THEN ...;	=	if (a == b) { /*...*/ }
Not equal	IF a <> b THEN ...;	!=	if (a != b) { /*...*/ }
Comparisons	IF a > b / a < b / a >= b / a <= b THEN ...;	>, <, >=, <=	if (a > b) { } if (a < b) { } if (a >= b) { } if (a <= b) { }
BETWEEN	IF salary BETWEEN 30000 AND 50000 THEN ...;	min <= x && x <= max	if (salary >= 30000 && salary <= 50000) { /*...*/ }
IN list	SET Result = region IN('EAST','WEST','NORTH');	Set.contains	boolean result = java.util.Set.of("EAST", "WEST", "NORTH").contains(region);
IS NULL	IF amount IS NULL THEN SET amount = 0; END IF;	null check	if (amount == null) { amount = 0; }
LIKE	IF Body.Trade.Company LIKE 'I__' THEN SET flag = TRUE; END IF;	String.matches	if (company != null && company.matches("I..")) { flag = true; }
AND/OR/NO T	IF A AND B / A OR B / NOT flag THEN ...;	&&, , !	if (A && B) { } if (A B) { } if (!flag) { }

Add	SET total = price + tax;	+	double total = price + tax;
Subtract	SET diff = a - b;	-	int diff = a - b;
Multiply	SET result = x * y;	*	int result = x * y;
Divide	SET avg = sum / count;	/	double avg = (double) sum / count;
Concat	SET msg = 'Hello ' 'World';	String +	String msg = "Hello " + "World";
Block	BEGIN ATOMIC DECLARE x INTEGER; SET x = 100; END;	Java block	{ int x = 100; }
CALL	CALL ComputeTax(InputRoot.XMLNSC.Order, OutputRoot.XMLNSC.OrderTax);	method call	computeTax(inputOrder, outputOrderTax);
CASE	CASE WHEN amount > 1000 THEN ... END CASE;	if / else-if	if (amount > 1000) { status = "High"; } else if (amount > 0) { status = "Medium"; } else { status = "Low"; }
CREATE FUNCTION	CREATE FUNCTION GetDiscount(...) RETURNS DECIMAL ...;	static method	static BigDecimal getDiscount(BigDecimal orderValue){ return orderValue.multiply(new BigDecimal("0.05")); }
CREATE PROCEDURE	CREATE PROCEDURE ProcessOrder(IN refOrder REFERENCE) ...;	void method	void processOrder(Order refOrder) { /* logic */ }
IF/ELSE	IF count = 0 THEN SET status = 'None'; ELSE SET status = 'Some'; END IF;	if / else	if (count == 0) { status = "None"; } else { status = "Some"; }
WHILE	WHILE i < CARDINALITY(...) DO ... END WHILE;	while loop	while (i < items.size()) { /*...*/ i++; }
REPEAT	REPEAT ... UNTIL i >= 10 END REPEAT;	do/while	do { i++; sum += i; } while (i < 10);
RETURN	RETURN resultValue;	return	return resultValue;
THROW USER EXCEPTION	THROW USER EXCEPTION ... VALUES(orderId);	throw custom exception	throw new OrderException("OrderErrors", 1001, orderId);
INSERT	INSERT INTO Database.Table (ID, NAME) VALUES (1, 'ABC');	JDBC INSERT	try (var ps = conn.prepareStatement("INSERT INTO Table(ID,NAME) VALUES(?,?)") { ps.setInt(1,1); ps.setString(2,"ABC"); ps.executeUpdate(); }
UPDATE	UPDATE Database.Table SET Status = 'OK' WHERE ID = 1;	JDBC UPDATE	try (var ps = conn.prepareStatement("UPDATE Table SET Status=? WHERE ID=?") { ps.setString(1,"OK"); ps.setInt(2,1); ps.executeUpdate(); }
DELETE	DELETE FROM Database.Table WHERE Status = 'CLOSED';	JDBC DELETE	try (var ps = conn.prepareStatement("DELETE FROM Table WHERE Status=?") { ps.setString(1,"CLOSED"); ps.executeUpdate(); }
SQLCODE	SET sqlCode = SQLCODE;	SQLException.getErrorCode()	try { /* db */ } catch (java.sql.SQLException e) { int sqlCode = e.getErrorCode(); }
SQLSTATE	SET state = SQLSTATE;	SQLException.getSQLState()	try { /* db */ } catch (java.sql.SQLException e) { String state = e.getSQLState(); }
SQLERRORTXT	SET errText = SQLERRORTXT;	SQLException.getMessage()	try { /* db */ } catch (java.sql.SQLException e) { String errText = e.getMessage(); }
LOG	LOG USER 'Order processed: ' orderId;	logger	log.info("Order processed: {}", orderId);
CURRENT_DATE	SET Result = CURRENT_DATE;	LocalDate.now()	java.time.LocalDate result = java.time.LocalDate.now();
CURRENT_TIME	SET Result = CURRENT_TIME;	LocalTime.now()	java.time.LocalTime result = java.time.LocalTime.now();
CURRENT_TIMESTAMP	SET Result = CURRENT_TIMESTAMP;	LocalDateTime.now()	java.time.LocalDateTime result = java.time.LocalDateTime.now();
ABS	SET R = ABS(-5);	Math.abs	int R = Math.abs(-5);
ACOS	SET R = ACOS(0.5);	Math.acos	double R = Math.acos(0.5);
MOD	SET R = MOD(10, 3);	%	int R = 10 % 3;
POWER	SET R = POWER(2, 3);	Math.pow	double R = Math.pow(2.0, 3.0);
SQRT	SET R = SQRT(16);	Math.sqrt	double R = Math.sqrt(16.0);
CONTAINS	CONTAINS('Hello World','ello')	String.contains	boolean r = "Hello World".contains("ello");
LENGTH	LENGTH('Hello')	String.length	int len = "Hello".length();
LOWER	LOWER('HELLO')	toLowerCase	String r = "HELLO".toLowerCase(java.util.Locale.ROOT);
UPPER	UPPER('abc')	toUpperCase	String r = "abc".toUpperCase(java.util.Locale.ROOT);
TRIM	TRIM(' ABC ')	strip	String r = " ABC ".strip();
URLENCODE	URLENCODE('Hello World')	URLEncoder.encode	String r = java.net.URLEncoder.encode("Hello World", java.nio.charset.StandardCharsets.UTF_8);
URLDECODE	URLDECODE('Hello%20World')	URLDecoder.decode	String r = java.net.URLDecoder.decode("Hello%20World", java.nio.charset.StandardCharsets.UTF_8);
BASE64ENC	SET x = BASE64ENCODE('ABC');	Base64.getEncoder	String x = java.util.Base64.getEncoder().encodeToString("ABC".getBytes(java.nio.charset.StandardCharsets.UTF_8));

BASE64DECODE	SET x = BASE64DECODE('QUIJD');	Base64.getDecoder	byte[] x = java.util.Base64.getDecoder().decode("QUIJD");
COALESCE	SET X = COALESCE(NULL, 'ABC');	Objects.requireNonNullElse	String x = java.util.Objects.requireNonNullElse(null, "ABC");
SLEEP	CALL SLEEP(5000);	Thread.sleep	try { Thread.sleep(5000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
UUIDASCHAR	SET x = UUIDASCHAR();	UUID.toString	String x = java.util.UUID.randomUUID().toString();

Hasta este punto se ha evaluado los nodos y el lenguaje base de la tecnología a migrar, sin embargo, esta pendiente el tipo de mensajería, esto determinara en la estructura de clean arquitectura (remítase en la ilustración 3) como las entidades. En este caso, el sector bancario usa a menudo los siguientes arquetipos:

1. Extensible Markup Language (XML).
2. JavaScript Object Notation (json).
3. Y trama plana.

En el caso del primero, se pasa el prompt para corroborar la creación de las entidades correspondientes y considerando la diferenciación entre las capas, también debe manejar sus respectivos data transfer object (DTO). El material de insumo es tomado desde la documentación de IIB10 (IBM, 2022), acoplado al PROMPT 22:

PROMPT 22

According the next code: CREATE COMPUTE MODULE xmlns_doc_flow_Compute CREATE FUNCTION Main() RETURNS BOOLEAN BEGIN CALL CopyMessageHeaders(); -- Declaration of namespace constants -- These are only used by ESQl DECLARE sp1 NAMESPACE 'http://www.ibm.com/space1'; DECLARE sp2 NAMESPACE 'http://www.ibm.com/space2'; DECLARE sp3 NAMESPACE 'http://www.ibm.com/space3'; -- Namespace declaration for prefix 'space1' SET OutputRoot.XMLNS.message.(XML.NamespaceDecl)xmlns:space1 = 'http://www.ibm.com/space1'; SET OutputRoot.XMLNS.message.sp1:data1 = 'Hello!'; -- Default namespace declaration (empty prefix) SET OutputRoot.XMLNS.message.sp2:data2.(XML.NamespaceDecl)xmlns = 'http://www.ibm.com/space2'; SET OutputRoot.XMLNS.message.sp2:data2.sp2:subData1 = 'Hola!'; SET OutputRoot.XMLNS.message.sp2:data2.sp2:subData2 = 'Guten Tag!'; SET OutputRoot.XMLNS.message.sp3:data3 = 'Bonjour!'; RETURN TRUE; END; CREATE

PROCEDURE CopyMessageHeaders() BEGIN DECLARE I INTEGER 1; DECLARE J INTEGER CARDINALITY(InputRoot.*[I]); WHILE I < J DO SET OutputRoot.*[I] = InputRoot.*[I]; SET I = I + 1; END WHILE; END; END MODULE; Generate objects related to this in Java using the Spring Boot framework and Lombok include DTOs and entities. Resulting message in XML format and, additionally, illustrate the resulting objects in a class diagram.

Con lo anterior, el mensaje resultante con sus respectivas diagrama de clases queda plasmado en la Tabla 7
Resultado creación entidades y dtos desde código ESQl con énfasis en XML

Tabla 7
Resultado creación entidades y dtos desde código ESQl con énfasis en XML

Resulting message	Class diagram
<pre><message xmlns:space1="http://www.ibm.com/space1"> <space1:data1>Hello!</space1:data1> <data2 xmlns="http://www.ibm.com/space2"> <subData1>Hola!</subData1> <subData2>Guten Tag!</subData2> </data2> <data3 xmlns="http://www.ibm.com/space3">Bonjour!</data3> </message></pre>	<pre>classDiagram class MessageService { -MessageMapper mapper +MessageXml createMessage() } class MessageMapper { +MessageXml toXml(MessageDto dto) +Data2Xml toXml(Data2Dto dto) } class MessageDto { -String data1 -Data2Dto data2 -String data3 } class MessageXml { +String SPACE1 +String SPACE2 +String SPACE3 -String data1 -Data2Xml data2 -String data3 } class Data2Dto { -String subData1 -String subData2 } class Data2Xml { -String subData1 -String subData2 } MessageService --> MessageMapper MessageMapper --> MessageDto MessageMapper --> MessageXml MessageMapper --> Data2Dto MessageMapper --> Data2Xml MessageDto --> Data2Dto</pre>

Continuando con el proceso, en este caso se aplica el siguiente prompt para la mensajería json y su respective resultado en la Tabla 8:

PROMPT 23

According the next code: DECLARE myRow ROW; SET myRow.rowData[1].fieldOne = 'Row1Field1'; SET myRow.rowData[1].fieldTwo = 'Row1Field2'; SET

myRow.rowData[2].fieldOne = 'Row2Field1'; SET myRow.rowData[2].fieldTwo = 'Row2Field2'; SET OutputRoot.JSON.Data.aRow = myRow; Generate objects related to this in Java using the Spring Boot framework and Lombok include DTOs and entities. Resulting message in JSON format and, additionally, illustrate the resulting objects in a class diagram.

Tabla 8
Resultado creación entidades y dtos desde código ESQL con énfasis en XML

Resulting message	Class diagram
<pre> { "Data": { "aRow": { "rowData": [{ "fieldOne": "Row1Field1", "fieldTwo": "Row1Field2" }, { "fieldOne": "Row2Field1", "fieldTwo": "Row2Field2" }] } } } </pre>	<pre> classDiagram class RowMessageController { -RowMessageService rowMessageService +JsonResponseDto getRows() } class RowMessageService { -RowMapper rowMapper +JsonResponseDto createMessage() } class RowMapper { +JsonResponseDto toResponseDto(ARowEntity entity) +ARowDto toDto(ARowEntity entity) } class JsonResponseDto { -DataDto Data } class ARowEntity { -Long id -List<RowDataEntity> rowData } class DataDto { -ARowDto aRow } class ARowDto { -List<RowDataDto> rowData } class RowDataEntity { -Long id -String fieldOne -String fieldTwo -ARowEntity aRow } class RowDataDto { -String fieldOne -String fieldTwo } RowMessageController --> RowMessageService RowMessageService --> RowMapper RowMapper --> JsonResponseDto RowMapper --> ARowEntity JsonResponseDto --> DataDto ARowEntity --> RowDataEntity DataDto --> ARowDto ARowEntity --> ARowDto ARowEntity --> RowDataEntity RowDataEntity --> RowDataDto </pre>

En última instancia, para la mensajería en texto plano, tiene un tratamiento especial, dado que, se debe contar con otro archivo en la tecnología IIB10.

10.2 Creación prompts lenguaje a migrar

Con respecto a la Ilustración 3, se centrará inicialmente en los entryPoints y gateways. Estos son los que interactuarán con otros servicios y/o expondrán los servicios, ejemplo: API Rest, MQ, SOAP, SFTP, etc. Para ello, también se hará uso de la Tabla 5, sobre todo, en los que son tipo input y output, con el fin de hacer la migración desde este punto de partida.

Considerando lo mencionado previamente, se extrae los posibles valores para los entryPoints desde la documentación de la arquitectura limpia en scaffold, con el siguiente PROMPT 24:

PROMPT 24

Create a list of instructions based on the table shown at the following URL, regarding "Generate Entry Point." <https://bancolombia.github.io/scaffold-clean-architecture/docs/tasks/generate-entry-point>. Consider the following: 1. In the following command: `gradle generate EntryPoint -type=[entryPointType]`, replace `[entryPointType]` with the value found in the "Type" column. 2. If the parameter contains a hyphen, the command should extend up to the "Type" column. 3. Create a separate command for each parameter, specifying for each one a value listed in the "Values" column. 4. Create table in excel.

La respuesta anterior, queda concisa en la Tabla 9.

Tabla 9
Combinaciones comandos entryPoint

Command
<code>gradle generateEntryPoint --type=generic --name=<String></code>
<code>gradle generateEntryPoint --type=asynceventhandler --eda=true</code>
<code>gradle generateEntryPoint --type=asynceventhandler --eda=false</code>
<code>gradle generateEntryPoint --type=asynceventhandler --tech=rabbitmq</code>
<code>gradle generateEntryPoint --type=asynceventhandler --tech=kafka</code>
<code>gradle generateEntryPoint --type=asynceventhandler --tech=rabbitmq</code>
<code>gradle generateEntryPoint --type=asynceventhandler --tech=kafka</code>
<code>gradle generateEntryPoint --type=graphql --pathgql=<String (path)></code>
<code>gradle generateEntryPoint --type=kafka</code>
<code>gradle generateEntryPoint --type=mcp --name=<String></code>
<code>gradle generateEntryPoint --type=mcp --enable-tools=true</code>
<code>gradle generateEntryPoint --type=mcp --enable-tools=false</code>

gradle generateEntryPoint --type=mcp --enable-resources=true
gradle generateEntryPoint --type=mcp --enable-resources=false
gradle generateEntryPoint --type=mcp --enable-prompts=true
gradle generateEntryPoint --type=mcp --enable-prompts=false
gradle generateEntryPoint --type=mcp --enable-security=true
gradle generateEntryPoint --type=mcp --enable-security=false
gradle generateEntryPoint --type=mcp --enable-audit=true
gradle generateEntryPoint --type=mcp --enable-audit=false
gradle generateEntryPoint --type=mq
gradle generateEntryPoint --type=restmvc --server=tomcat
gradle generateEntryPoint --type=restmvc --server=jetty
gradle generateEntryPoint --type=restmvc --authorization=true
gradle generateEntryPoint --type=restmvc --authorization=false
gradle generateEntryPoint --type=restmvc --versioning=HEADER
gradle generateEntryPoint --type=restmvc --versioning=PATH
gradle generateEntryPoint --type=restmvc --versioning=NONE
gradle generateEntryPoint --type=restmvc --from-swagger=<File path>
gradle generateEntryPoint --type=restmvc --swagger=true
gradle generateEntryPoint --type=restmvc --swagger=false
gradle generateEntryPoint --type=rsocket
gradle generateEntryPoint --type=sqs
gradle generateEntryPoint --type=webflux --router=true
gradle generateEntryPoint --type=webflux --router=false
gradle generateEntryPoint --type=webflux --authorization=true
gradle generateEntryPoint --type=webflux --authorization=false
gradle generateEntryPoint --type=webflux --versioning=HEADER
gradle generateEntryPoint --type=webflux --versioning=PATH
gradle generateEntryPoint --type=webflux --versioning=NONE
gradle generateEntryPoint --type=webflux --from-swagger=<File path>
gradle generateEntryPoint --type=webflux --swagger=true
gradle generateEntryPoint --type=webflux --swagger=false
gradle generateEntryPoint --type=kafkastrimzi --name=<String>
gradle generateEntryPoint --type=kafkastrimzi --topic-consumer=<String (topic name)>
gradle generateEntryPoint --type=agent --name=<String>
gradle generateEntryPoint --type=agent --agent-enable-kafka=true
gradle generateEntryPoint --type=agent --agent-enable-kafka=false
gradle generateEntryPoint --type=agent --agent-enable-mcp-client=true
gradle generateEntryPoint --type=agent --agent-enable-mcp-client=false

Ahora, para los gateways, también conocido como drivenAdapters, en el plugin de scaffold. Su correspondiente prompt es:

Create a list of instructions based on the table shown at the following URL, regarding "Generate Entry Point." <https://bancolombia.github.io/scaffold-clean-architecture/docs/tasks/generate-driven-adapter>. Consider the following: 1. In the following command: `gradle generateDrivenAdapter--type=[drivenAdapterType]`, replace `[drivenAdapterType]` with the value found in the "Type" column. 2. If the parameter contains a hyphen, the command should extend up to the "Type" column. 3. Create a separate command for each parameter, specifying for each one a value listed in the "Values" column. 4. Create table in excel.

Su concerniente respuesta se encuentra concisa en la Tabla 10.

Tabla 10
Combinaciones comandos entryPoint

Command
<code>gradle generateDrivenAdapter --type=generic --name=<String></code>
<code>gradle generateDrivenAdapter --type=asynceventbus --eda=true</code>
<code>gradle generateDrivenAdapter --type=asynceventbus --eda=false</code>
<code>gradle generateDrivenAdapter --type=asynceventbus --tech=rabbitmq</code>
<code>gradle generateDrivenAdapter --type=asynceventbus --tech=kafka</code>
<code>gradle generateDrivenAdapter --type=asynceventbus --tech=rabbitmq,kafka</code>
<code>gradle generateDrivenAdapter --type=binstash --cache-mode=LOCAL</code>
<code>gradle generateDrivenAdapter --type=binstash --cache-mode=CENTRALIZED</code>
<code>gradle generateDrivenAdapter --type=binstash --cache-mode=HYBRID</code>
<code>gradle generateDrivenAdapter --type=cognitotokenprovider</code>
<code>gradle generateDrivenAdapter --type=dynamodb</code>
<code>gradle generateDrivenAdapter --type=jpa --secret=true</code>
<code>gradle generateDrivenAdapter --type=jpa --secret=false</code>
<code>gradle generateDrivenAdapter --type=kms</code>
<code>gradle generateDrivenAdapter --type=mongodb --secret=true</code>
<code>gradle generateDrivenAdapter --type=mongodb --secret=false</code>
<code>gradle generateDrivenAdapter --type=mq</code>
<code>gradle generateDrivenAdapter --type=r2dbc</code>
<code>gradle generateDrivenAdapter --type=redis --mode=template</code>
<code>gradle generateDrivenAdapter --type=redis --mode=repository</code>
<code>gradle generateDrivenAdapter --type=redis --secret=true</code>
<code>gradle generateDrivenAdapter --type=redis --secret=false</code>
<code>gradle generateDrivenAdapter --type=restconsumer --url=<URL></code>
<code>gradle generateDrivenAdapter --type=restconsumer --from-swagger=<file-path></code>
<code>gradle generateDrivenAdapter --type=rsocket</code>

<code>gradle generateDrivenAdapter --type=s3</code>
<code>gradle generateDrivenAdapter --type=secrets --secrets-backend=aws_secrets_manager</code>
<code>gradle generateDrivenAdapter --type=secrets --secrets-backend=vault</code>
<code>gradle generateDrivenAdapter --type=secretskafkastrimzi --secret-name=<String></code>
<code>gradle generateDrivenAdapter --type=sqs</code>

10.3 Evaluación modelos para la migración de código

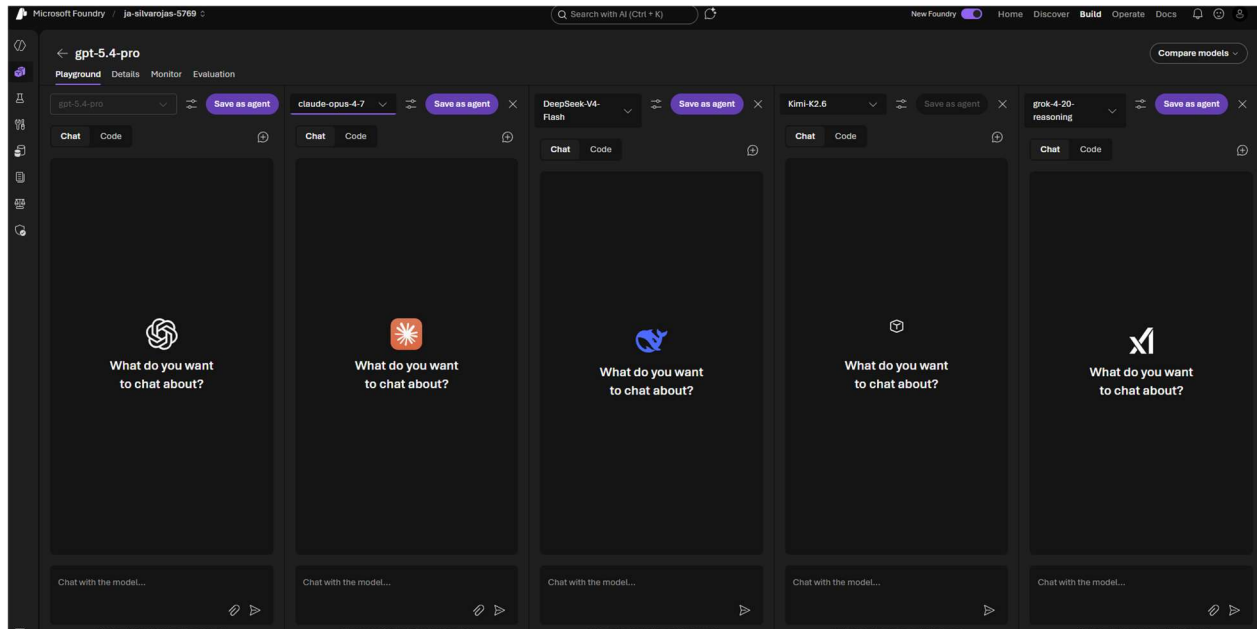
De acuerdo a lo mencionado en (Analysis, 2026), se usan los mejores cinco modelos de LLM; esta información se contrasta con la tabla de clasificación entregada por la herramienta de Microsoft Foundry (Microsoft Foundry, 2026). Para escoger los mejores LLM, se basa en el parámetro con nombre *índice de inteligencia artificial de análisis*. Este proporciona una puntuación única y medibles para evaluar y clasificar las capacidades de los LLM en cuatro ítems, a saber: agentes, codificación, razonamiento científico y capacidad general. Descrito lo anterior, se agrupan los seleccionados en la Tabla 11, cabe aclarar, que se excluye aquellos modelos que coincidan con el mismo propietario, en aras de contar con un abanico amplio de LLMs.

Tabla 11
Modelos LLM a evaluar

Name LLM	Versión
ChatGPT	5.4-pro
Claude Opus	4.7
Gemini	3.1 Pro
Kimi	K2.6
DeepSeek	V4-Pro
Mimo	V2.5-pro
Grok	4-20

Para validar los modelos, se emplea los prompts descritos en los apartes 10.1 y 10.2, además, como herramienta para determinar esta evaluación, se emplea (Microsoft Foundry, 2026), en particular, dos características que facilita esta tecnología, a saber: “Compare Models” y “Flow Prompts”.

Inicialmente, la herramienta tiene una limitante para realizar web scrapping(extracción de datos web) no lo permite, por políticas de derechos de autor, entre otras. Por lo que se hace necesario que se creen archivos intermedios con el contenido de los websites.



Parse the next xml to extract all properties from both the nodes and connections objects. For each node, if a translations element exists, decompose its sub-properties by splitting keys on the period (.) delimiter, and append these separated translation properties as individual columns in the output table.

<<Mensaje XML>> Se omite el contenido por lo extenso del mismo.

11 RESULTADOS Y DISCUSIÓN

El desarrollo de esta metodología estructura la migración de código legado con el uso de inteligencia artificial, en particular, LLM; facilitando este proceso tan complejo y costoso, permitiendo que varios sectores se vean beneficiados, entre ellos:

- Técnico: Facilitar al equipo desarrollador, el migrado de código, cumpliendo con la semántica del original y su funcionalidad, de igual forma, implementar o ajustar el código

a arquitecturas recientes. Reduciendo el tiempo de implementación y el esfuerzo requerido por el equipo en cuestión.

- Económico: Como se describió en el transcurso del documento, uno de los sectores con mayor código legado, es el sector bancario. En este, la ejecución de una metodología que cubra este campo, permite potenciar áreas como: mantenimiento, rendimiento y escalabilidad, que a corto y mediano plazo se verá reflejado en la disminución de costos en infraestructura.
- Seguridad: Cierre de brechas en este ítem, dado que, la migración permite resolver vulnerabilidades ya sea por desuso de tecnologías y/o falta de soporte.

12 Bibliografía

- Ziftci, C., Nikolov, S., Sjövall, A., Kim, B., Codecasa, D., & Kim, M. (2025). Migrating Code At Scale With LLMs At Google. *ACM International Conference on the Foundations of Software Engineering*, 12.
- Adamyk, B., Benson, V., Adamyk, O., Fraczek, B., & Chinnaswamy, A. (2024). Barriers to Trust in AI: a Study of the Explainability Technologies Adoption in Banks. *2024 14th International Conference on Advanced Computer Information Technologies*, 5.
- Alder, E. (2024, Febrero 20). *Simplify Your Complex Code Migration Projects With AI*. Retrieved from Simplify Your Complex Code Migration Projects With AI: https://dev.to/hackmamba/simplify-your-complex-code-migration-projects-with-ai-3fn8?utm_source=chatgpt.com
- Aleksyuk, A., & Itsykson, V. (2017). Automated Semantics-Driven Source Code Migration: a Pilot Prototype. *System Informatics*, 10.
- Amazon. (2024). *Amazon Web Services (AWS)*. Retrieved from Modelo de responsabilidad compartida: <https://aws.amazon.com/es/compliance/shared-responsibility-model/>
- Analysis, A. (2026, mayo 10). *LLM Leaderboard - Comparison of over 100 AI models from OpenAI, Google, DeepSeek & others*. Retrieved from <https://artificialanalysis.ai/>: https://artificialanalysis.ai/leaderboards/models?utm_source=chatgpt.com
- Aniche, M. (2022). *Effective Software Testing A developer's guide*. Shelter Island: Mannign Publications Co.

- Associates, A.-I. (2024). Leveraging AI to Modernize Legacy Code in Federal Civilian Agencies. *American Council for Technology-Industry Advisory Council*, 22.
- Bancolombia Tech. (2020, marzo 3). *medium.com*. Retrieved from Clean Architecture — Aislando los detalles: <https://medium.com/bancolombia-tech/clean-architecture-aislando-los-detalles-4f9530f35d7a>
- Barden Rubbo, F., Kessler Piveta, E., & Daltro, J. (2012). Inference rules for generic code migration of aspect-oriented programs. *Science of Computer Programming*, 19.
- Bennett, K., & Rajlich, V. (2000). Software Maintenance and Evolution: a Roadmap. *IEEE*, 14.
- Benoît, V., Abderrahmane, S., Nicolas, A., Anne, E., Stéphane, D., & Mustapha, D. (2021). Migrating GUI behavior: from GWT to Angular. *IEEE*, 10.
- Code Modernization: Benefits, Strategies, And AI Acceleration*. (2024, Agosto 10). Retrieved from <https://stefanini.com/>: https://stefanini.com/en/insights/news/code-modernization-benefits-strategies-and-ai-acceleration?utm_source=chatgpt.com
- Colombia, S. F. (2025, 03 26). *Indicadores de Seguridad de la Información y Ciberseguridad 2024*. Retrieved from <https://www.superfinanciera.gov.co/>: <https://www.superfinanciera.gov.co/publicaciones/10115571/indicadores-de-seguridad-de-la-informacion-y-ciberseguridad-2024/>
- De Marco, A., Iancu, V., & Asinofsky, I. (2018). COBOL to Java and Newspapers Still Get Delivered. *IEEE International Conference on Software Maintenance and Evolution*, 4.
- EUVIC. (2024). *Migration & Modernization of Legacy Applications: The 7 Steps Guide + Checklist*. Retrieved from <https://www.euvic.com/>: https://www.euvic.com/us/post/migration-of-legacy-applications-the-what-why-and-how/?utm_source=chatgpt.com
- Feathers, M. C. (2004). *Working Effectively with Legacy Code*. Pearson Education.
- Grupo Bancolombia. (2026). <https://bancolombia.github.io/>. Retrieved from Scaffold Clean Architecture: <https://bancolombia.github.io/scaffold-clean-architecture/docs/intro>
- Hall, M. (2023, Junio 01). *How the Banking Sector Impacts Our Economy*. Retrieved from <https://www.investopedia.com/>: <https://www.investopedia.com/ask/answers/032315/what-banking-sector.asp>

- Hughes, E., Hyland, R., Litvintchouk, S., Rosenthal, A., Shafer, A., & Surer, S. (1997). A methodology for migration of legacy applications to distributed object management. *IEEE*, 9.
- IBM. (2022). *www.ibm.com*. Retrieved from Working with namespaces: <https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=messages-working-namespaces>
- IBM. (2023, 06 21). *IBM Integration Bus*. Retrieved from *www.ibm.com*: <https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=product-overview>
- IBM. (2025, april). *What is legacy code?* Retrieved from IBM: <https://www.ibm.com/think/topics/legacy-code>
- Infante, C. (2010). *Guía para la presentación de proyectos de investigación*. Bogotá, Colombia: Universidad Nacional de Colombia.
- Isha, V., & Vishalakshmi, P. (2023). An Approach to Clean Architecture for Microservices using Python. *2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions*, 5.
- Ishaani, M., Behrooz, O.-T., & Anmol, A. (2024). Evaluating Human-AI Partnership for LLM-based Code Migration. *Amazon web Science*.
- Microsoft Foundry. (2026, May 18). *Model Leaderboard*. Retrieved from *ai.azure.com*: <https://ai.azure.com/nextgen>
- Mili, A., & Tchier, F. (2015). *Software Testing: Concepts and Operations*. New Jersey: Wiley.
- Oglietti, G. (2024, Agosto 15). *El sector bancario colombiano*. Retrieved from <https://www.celag.org/>: <https://www.celag.org/el-sector-bancario-colombiano/>
- República, C. d. (2022, 01 25). *Ley 1328 de 2009*. Retrieved from Función Pública: <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=36841>
- Sebesta, R. W. (2012). *Concepts or programming languages 10th edition*. Colorado: Pearson.
- Singireddy, D. (2025). System Integration in Financial Services: A Comprehensive Analysis of Modern Banking Architecture. *Sarcouncil Journal of Engineering and Computer Sciences*, 364-373.
- Svensson, N. (2018). Defining and Identifying Legacy Code in Software A case study developing 3D visual camera surveillance software . *Computer Science*, 34.
- Wayne Xin Zhao, K. Z. (2025). A Survey of Large Language Models. 144.